```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Import image
image = cv2.imread("flower.png")

if image is None:
    print("Error: flower.png not found!")
    raise SystemExit

# Resize image to reduce memory usage
h, w = image.shape[:2]
scale = min(800 / w, 800 / h, 1.0)
image = cv2.resize(
    image, (int(w * scale), int(h * scale))
)

# Convert BGR to RGB
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Function to display images
def show_images(images, titles, cmaps=None):
    if cmaps is None:
        cmaps = [None] * len(images)

    fig, axes = plt.subplots(
        1, len(images), figsize=(6 * len(images), 5)
    )
    axes = np.atleast_1d(axes)

    for ax, img, title, cmap in zip(
        axes, images, titles, cmaps
    ):
        ax.imshow(img, cmap=cmap)
        ax.set_title(title)
        ax.axis("off")

    plt.tight_layout()
    plt.show()

# 2. Display original image
show_images([image], ["Original Colour Image"])

# 3. Red intensity and yellow intensity
red = image.copy()
yellow = image.copy()

red[:, :, 1] = 0
red[:, :, 2] = 0

yellow[:, :, 2] = 0

show_images(
    [red, yellow],
    ["Red Intensity", "Yellow Intensity"]
)

# 4. Convert RGB image to grayscale
gray_image = cv2.cvtColor(
    image, cv2.COLOR_RGB2GRAY
)
```

```
print("Colored image shape:", image.shape)
print("Grayscale image shape:", gray_image.shape)

show_images(
    [image, gray_image],
    ["Color", "Grayscale"],
    [None, "gray"]
)

# 5. Convert grayscale image to binary using Otsu
threshold, binary_image = cv2.threshold(
    gray_image, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

show_images(
    [gray_image, binary_image],
    ["Grayscale", "Otsu Binary"],
    ["gray", "gray"]
)

# 6. Blur the grayscale image
blurred_image = cv2.GaussianBlur(
    gray_image, (0, 0), sigmaX=20
)

show_images(
    [gray_image, blurred_image],
    ["Gray Image", "20 Sigma Blur"],
    ["gray", "gray"]
)

# 7. Plot RGB colour histograms
colors = ["red", "green", "blue"]

plt.figure(figsize=(8, 5))

for i, color in enumerate(colors):
    plt.hist(
        image[:, :, i].ravel(),
        bins=256,
        range=(0, 256),
        color=color,
        alpha=0.45,
        label=color.capitalize()
    )

plt.title("RGB Colour Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.legend()
plt.tight_layout()
plt.show()

# 8. Calculate grayscale histogram
histogram = cv2.calcHist(
    [gray_image], [0], None, [256], [0, 256]
)

plt.figure(figsize=(8, 5))
plt.plot(histogram)
plt.title("Grayscale Histogram")
```

```
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.xlim([0, 256])
plt.tight_layout()
plt.show()

# 9. Extract image features
print("\n--- Extracted Image Features ---")
print("Red Mean:", round(np.mean(image[:, :, 0]), 2))
print("Green Mean:", round(np.mean(image[:, :, 1]), 2))
print("Blue Mean:", round(np.mean(image[:, :, 2]), 2))
print("Mean Intensity:", round(np.mean(gray_image), 2))
print("Standard Deviation:", round(np.std(gray_image), 2))
print("Contrast:", int(gray_image.max()) - int(gray_image.min()))

print("\nFeature extraction and histogram plotting completed!")
```

Output:-

```
G:\BE_Practical\ISR\ImageFeatureExtraction>python -m pip install opencv-python numpy
matplotlib

G:\BE_Practical\ISR\ImageFeatureExtraction>cd /d
G:\BE_Practical\ISR\ImageFeatureExtraction

G:\BE_Practical\ISR\ImageFeatureExtraction>set OPENBLAS_NUM_THREADS=1

G:\BE_Practical\ISR\ImageFeatureExtraction>set OMP_NUM_THREADS=1

G:\BE_Practical\ISR\ImageFeatureExtraction>python feature_extraction.py
Colored image shape: (600, 800, 3)
Grayscale image shape: (600, 800)

--- Extracted Image Features ---
Red Mean: 142.54
Green Mean: 103.54
Blue Mean: 79.21
Mean Intensity: 112.43
Standard Deviation: 42.01
Contrast: 247

Feature extraction and histogram plotting completed!
```